



Artificial Intelligence 101

DeepFace: Closing the Gap to Human-Level Performance in Face Verification

CtrlC + CtrlV Crew

Pre-Requisites

MACHINE LEARNING

DEEP NEURAL NETWORKS

Support Vector
Machines

TRADITIONAL
NEURAL NETWORK
CONCEPTS

Activation
Functions:
ReLU

Back
Propagation

Training
Processes

Stochastic
Gradient
Descent

Loss Functions

Cross Entropy
Loss

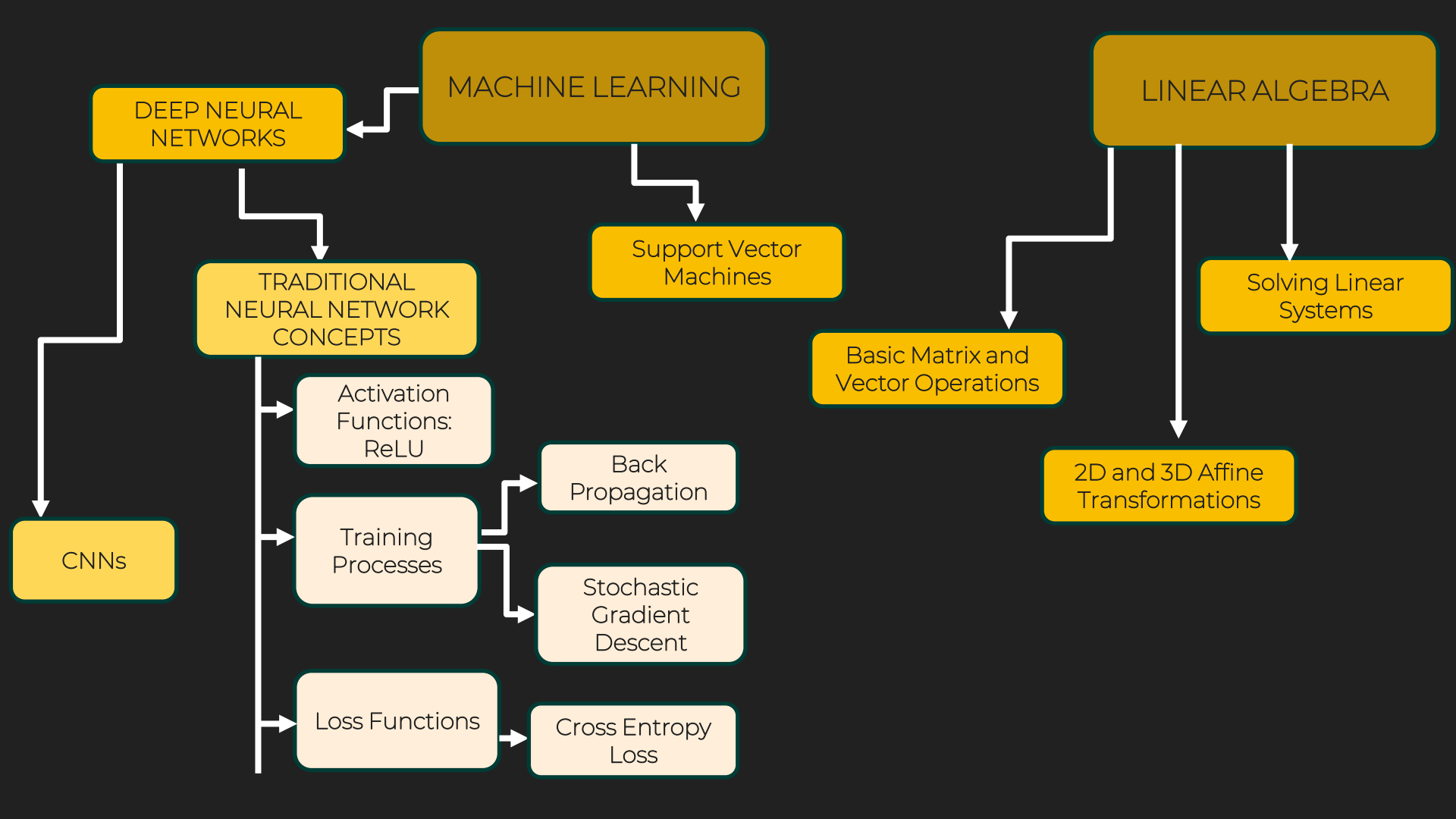
CNNs

LINEAR ALGEBRA

Basic Matrix and
Vector Operations

Solving Linear
Systems

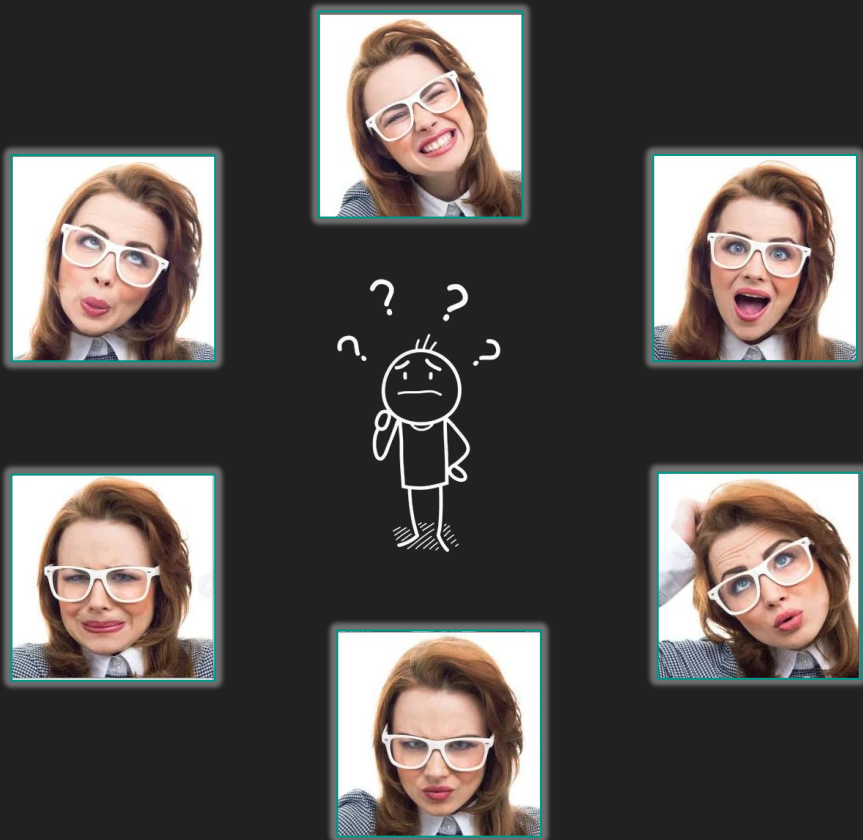
2D and 3D Affine
Transformations



What is it trying
to solve?



Determining if all these photos depict the same person



Input Images



Variations



Increased Complexity



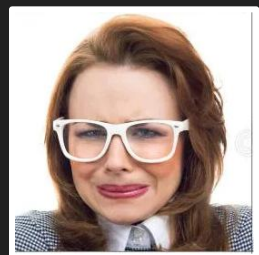
Variation in:



Pose



Lighting



Expressions





DeepFace is trying to solve the problem of :

1. face verification in unconstrained environments
2. variations in pose
3. Change in lighting
4. Variations in expression
5. Poor Image quality

And of course, matching the human level accuracy!

Early Approaches and their failures

Hand Crafted Models

```
graph TD; H[Hand Crafted Models] --> LBP[LBP]; H --> HOG[HOG]; LBP --> D1[Describes local texture by encoding relationships between a pixel and its neighbors.]; HOG --> D2[It works by dividing an image into small cells, calculating the gradient directions of pixel intensities within each cell, and creating histograms to represent these gradients]; W[Why They Failed?] --> P[Pose Sensitivity]; W --> L[Lighting Issues]; W --> R[Rigid Grid-Based Structure]; P --> D3[don't capture the spatial relationships between distant parts of the face (e.g., eyes and mouth). Faces in different poses lead to mismatched features.]; L --> D4[highly dependent on pixel intensities, which change drastically under different lighting conditions.]; R --> D5[HOG relies on dividing the image into fixed-size cells or blocks to compute gradient histograms. So it does not adapt to variations in face size, scale, or alignment, making the feature extraction process inconsistent.];
```

LBP

Describes local texture by encoding relationships between a pixel and its neighbors.

HOG

It works by dividing an image into small cells, calculating the gradient directions of pixel intensities within each cell, and creating histograms to represent these gradients

Why They Failed?

Pose Sensitivity

don't capture the spatial relationships between distant parts of the face (e.g., eyes and mouth). Faces in different poses lead to mismatched features.

Lighting Issues

highly dependent on pixel intensities, which change drastically under different lighting conditions.

Rigid Grid-Based Structure

HOG relies on dividing the image into fixed-size cells or blocks to compute gradient histograms. So it does not adapt to variations in face size, scale, or alignment, making the feature extraction process inconsistent.

Methods like SVMs, PCA, and LDA didn't work well because:

Not Adaptable

- Too focused on specific datasets
- Hard to work well with new or diverse data.

Lack of Depth

- Don't automatically learn complex features from raw data, missing out on details to improve accuracy.

High Dimensionality

- Used thousands of features, making systems computationally expensive.
- Inefficient for large-scale datasets.

Metric Learning Approaches

Joint Bayesian Model

A statistical model that jointly learns intra-personal and extra-personal variations (i.e., variations within the same person versus between different people).

Dataset Dependence

Joint Bayesian models are tuned to specific datasets and fail to generalize well to unseen conditions.

Feature Dependency

The performance depends entirely on the quality of the input features. Hand-crafted features like LBP or HOG, often used with this model, are insufficient in unconstrained environments.

Why It Failed?

All these methods individually didn't work appropriately
DeepFace succeeded in human-level performance because it combined many models together.

2D Face Alignment

FACE ALIGNMENT

1

Why do we need it

It's a crucial step in facial recognition, analysis and other computing vision tasks involving faces.

2

How do we achieve it

There are several alignment techniques available such as 2D alignment, 3D alignment, eye alignment, etc.

3

Challenges

There is no completely correct solution to it in the context of unconstrained face verification. Even 3D models are failing especially in unconstrained environment.

OPTIMIZATION USING 2D ALIGNMENT

STEP 1

Our goal is to map six key fiducial points from their original image positions to specific anchor positions using a 2D similarity transformation. This is transformed by

$$T_{2d} = \begin{bmatrix} sR & t \\ 0 & 1 \end{bmatrix}$$

STEP 2

The six fiducial points are transformed iteratively by:

$$x_j^{\text{anchor}} = T_{2d} \cdot x_j^{\text{source}}$$

The transformation is refined until the fiducial points stabilise

STEP 3

Finally we get

$$T_{2d}^{\text{final}} = T_{2d}^1 \cdot T_{2d}^2 \cdot \dots \cdot T_{2d}^k$$

This aggregated transformation generates a 2D aligned crop as shown below.



Optimizing DeepFace with 3D Face Alignment

Why is 3D Alignment necessary ?

Challenges with 2D alignment:

Handles only in-plane
transformations
(rotation, scale, translation)

Cannot correct
out-of-plane rotations

Solution with 3D alignment:

Incorporates depth information to map facial
landmarks accurately to a frontal pose

Steps in 3D Alignment



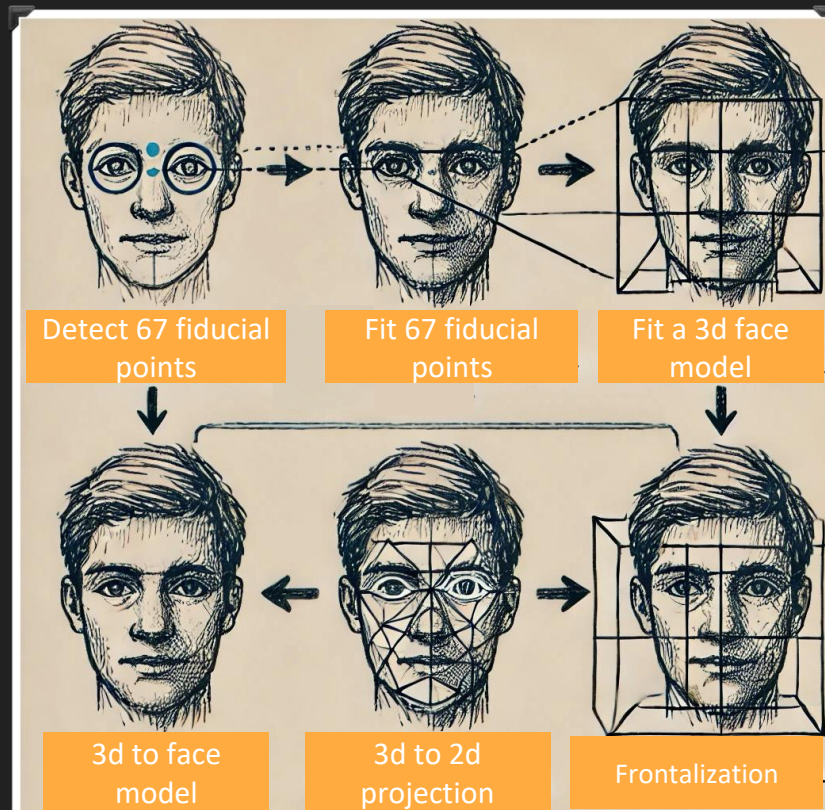
Detect 67 Fiducial Points:

- Locate detailed facial landmarks (e.g., eyes, nose, mouth, face contour).



Fit a Generic 3D Face Model:

- Align a 3D model to match the 2D fiducial points.



Frontalization:

- Warp the face to a frontal view using piecewise affine transformations.



3D-to-2D Projection:

- Use a camera projection model to map the 3D model to the 2D image plane.



Mathematics of 3D-Alignment

3D-to-2D Affine Camera
Projection

generalized
least squares solution

$$x_{2d} = X_{3d} \cdot P$$

- x_{2d} : 2D facial landmarks.
- X_{3d} : 3D anchor points on the generic model.
- P : Affine transformation matrix (2x4).

Residual Minimization

Minimize

$$\text{loss}(P) = r^T \Sigma^{-1} r, \quad r = x_{2d} - X_{3d} \cdot P$$

Rotation of the Face

- After finding the best projection matrix P , the 3D model is used to generate a frontalized version of the face.
- Any rotations (e.g., side profile or tilted head) are corrected during the frontalization step by reprojecting the 3D model into a frontal pose.

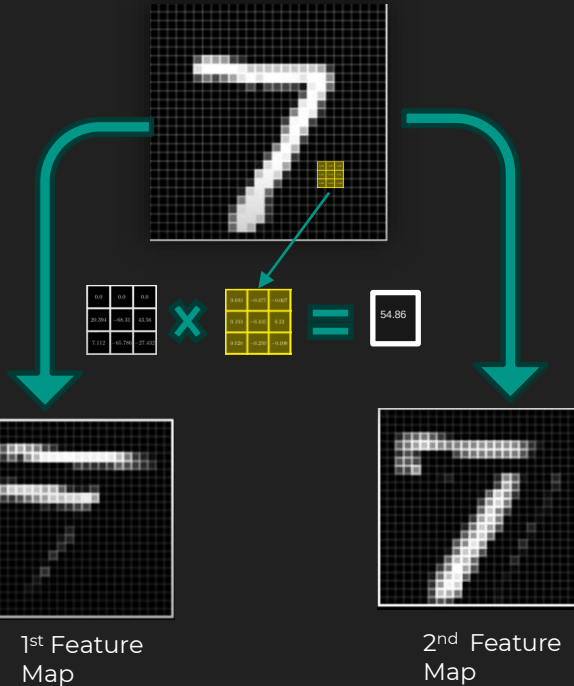
Mirroring
Symmetry

If certain regions of the face are not visible in the input image, the missing triangles are filled by mirroring symmetrical regions from the visible side.

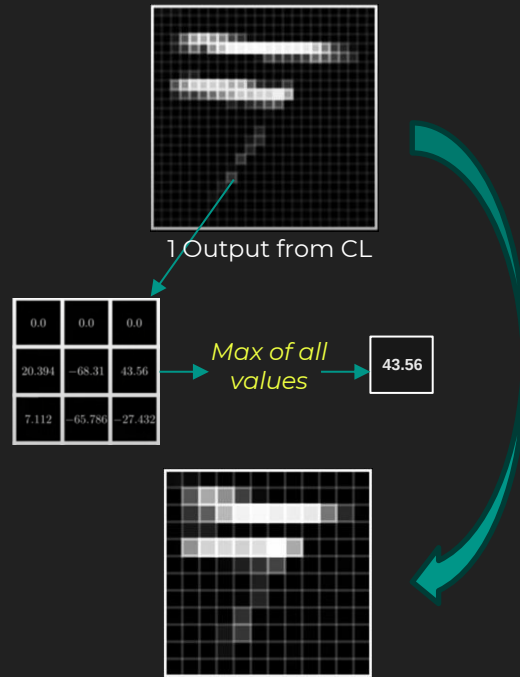
**What after
Frontalization?**

CONVOLUTIONAL NEURAL NETWORKS

CONVOLUTION LAYERS



MAX POOLING



LOCALLY CONNECTED

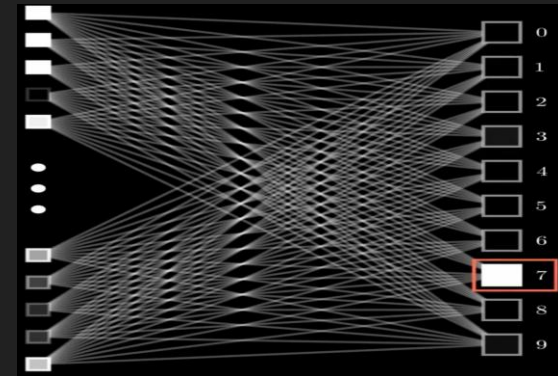
Same filter across the FM



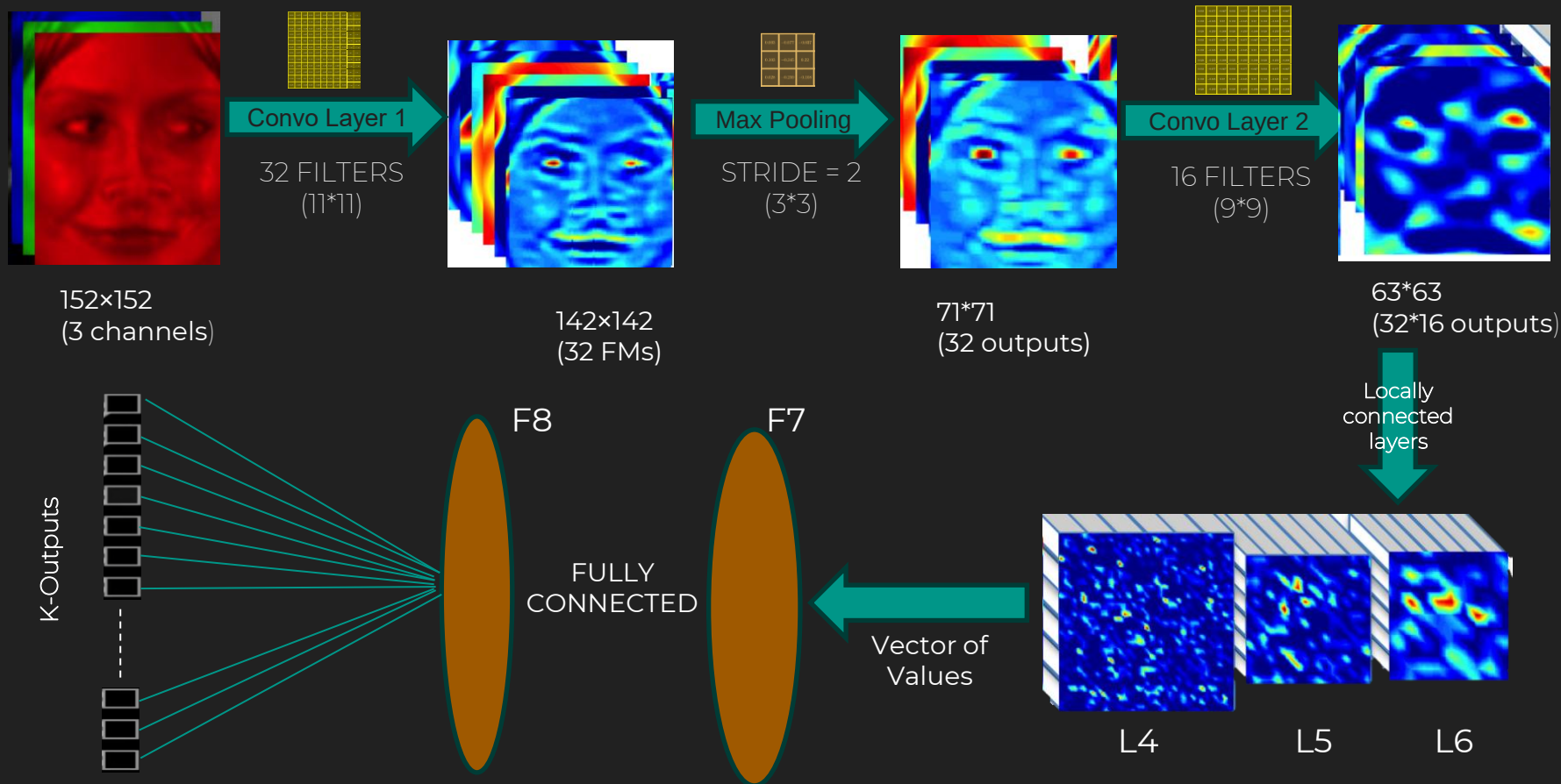
Different filters across the FM



FULLY CONNECTED



How DeepFace implemented this?



K-Way SoftMax

This produces a distribution of the data output from the last fully connected convolutional layer over class labels

o_k = the k -th output of the network on a given input. Then Probability assigned to the k -th class is the output of the softmax function:

$$p_k = \exp(o_k) / \sum_h \exp(o_h)$$

Loss Function and Stochastic Gradient Descent (SGD)

The Cross Entropy Loss for each training sample is minimized to increase the probability of getting the correct class.

Loss is: $L = -\log p_k$
Where k = index of the true label for a given input

The loss is minimized efficiently over the parameters by computing the gradient of L w.r.t. the parameters and by updating the parameters using Stochastic Gradient descent (SGD). The gradients are calculated through standard backpropagation of errors.

ReLu Activation Function

This is applied after every convolution on the locally and fully connected layers.
 $\text{ReLu} = \text{Max}(0, x)$

This produces non linear and sparse features and a regularization method called dropout sets the random features to 0 during the training.

$G(I)$ is the representation of the image I computed using a forward feed neural network as a composition of functions g_ϕ^ℓ for L layers.

$$G(I) = g_\phi^{F_7}(g_\phi^{L_6}(\dots g_\phi^{C_1}(T(I, \theta_T))\dots))$$

$$\phi = \{C_1, \dots, F_7\}$$

$$\theta_T = \{x_{2d}, \vec{P}, \vec{r}\}$$

Normalisation

Finally, the features are normalized between 0 and 1 This helps reduce the sensitivity of the model to variations in illumination, especially in applications like image processing.
Each component of the feature vector is divided by its largest value across the training set.

This is then followed by L_2 -normalization:

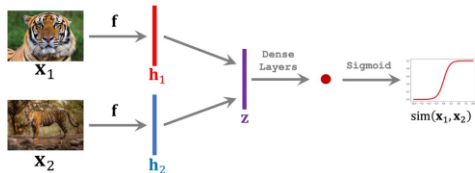
$$f(I) := \bar{G}(I) / \|\bar{G}(I)\|_2$$

$$\bar{G}(I)_i = G(I)_i / \max(G_i, \epsilon)$$

VERIFICATION METRIC

- ✓ Checks if two instances belong to the same identity.
- ✓ Inputs are often represented as feature vectors extracted by models like DeepFace in our paper or embeddings from neural networks.
- ✓ Supervised methods usually perform better than unsupervised ones, but training on one dataset (like a dataset of celebrity photos) can hurt performance on others.
- ✓ This work explores an unsupervised method using the inner product of feature vectors and also tests supervised methods like χ^2 similarity and Siamese networks for better generalization across datasets.

Siamese Network



We use the weighted- X^2 similarity to compare various DeepFace representations:

$$\chi^2(f_1, f_2) = \sum_i w_i (f_1[i] - f_2[i])^2 / (f_1[i] + f_2[i])$$

where f_1 and f_2 are the DeepFace representations. The weight parameters are learned using a linear SVM, applied to vectors of the elements. $(f_1[i] - f_2[i])^2 / (f_1[i] + f_2[i])$

Weighted X^2 distance

Supervised Verification Metric

Siamese network

It's an end-to-end metric learning approach. The face recognition network (without the top layer) is replicated twice (one for each input image) and the features are used to directly predict whether the two input images belong to the same person. The Siamese network's induced distance is:

$$d(f_1, f_2) = \sum_i \alpha_i |f_1[i] - f_2[i]|$$

where α_i are trainable parameters. The parameters of the Siamese network are trained by standard cross entropy loss and back-propagation of the error.



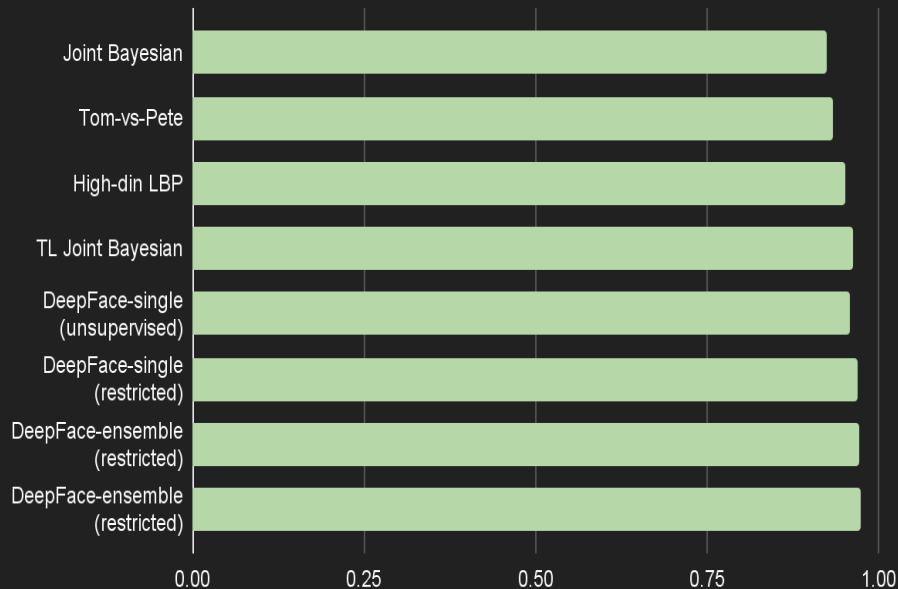
What DeepFace achieved?

Accuracy on LFW dataset

Recognition accuracy on the LFW dataset close to human performance (over 97.5%).

- ✓ Without frontalization (2D alignment), accuracy is 94.3%; without alignment, it drops to 87.9%.
- ✓ Using only frontalization and a simple classifier, accuracy reaches 91.4%.
- ✓ In an unsupervised setting, achieves 95.92% accuracy, and with a kernel SVM, accuracy improves to 97.00%, surpassing previous methods.

Comparison with LFW dataset

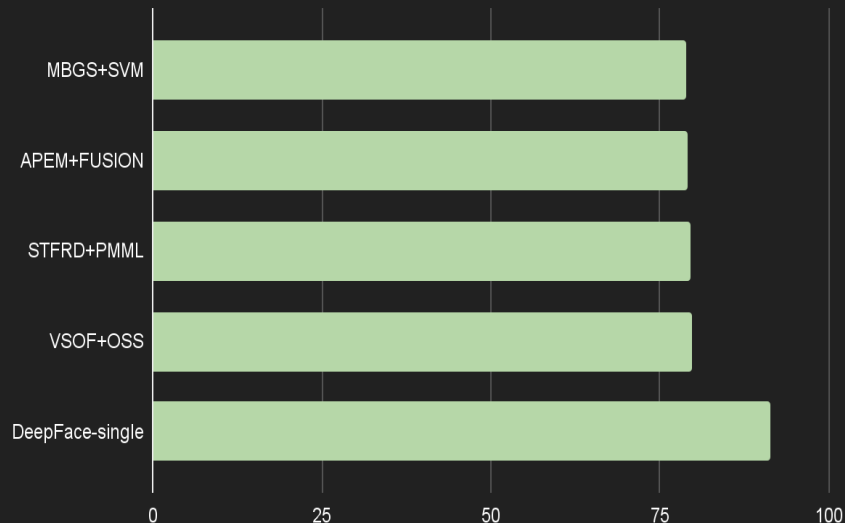


Accuracy on YTF dataset

Recognition accuracy on the YTF dataset was less achieving an accuracy of 91.4%

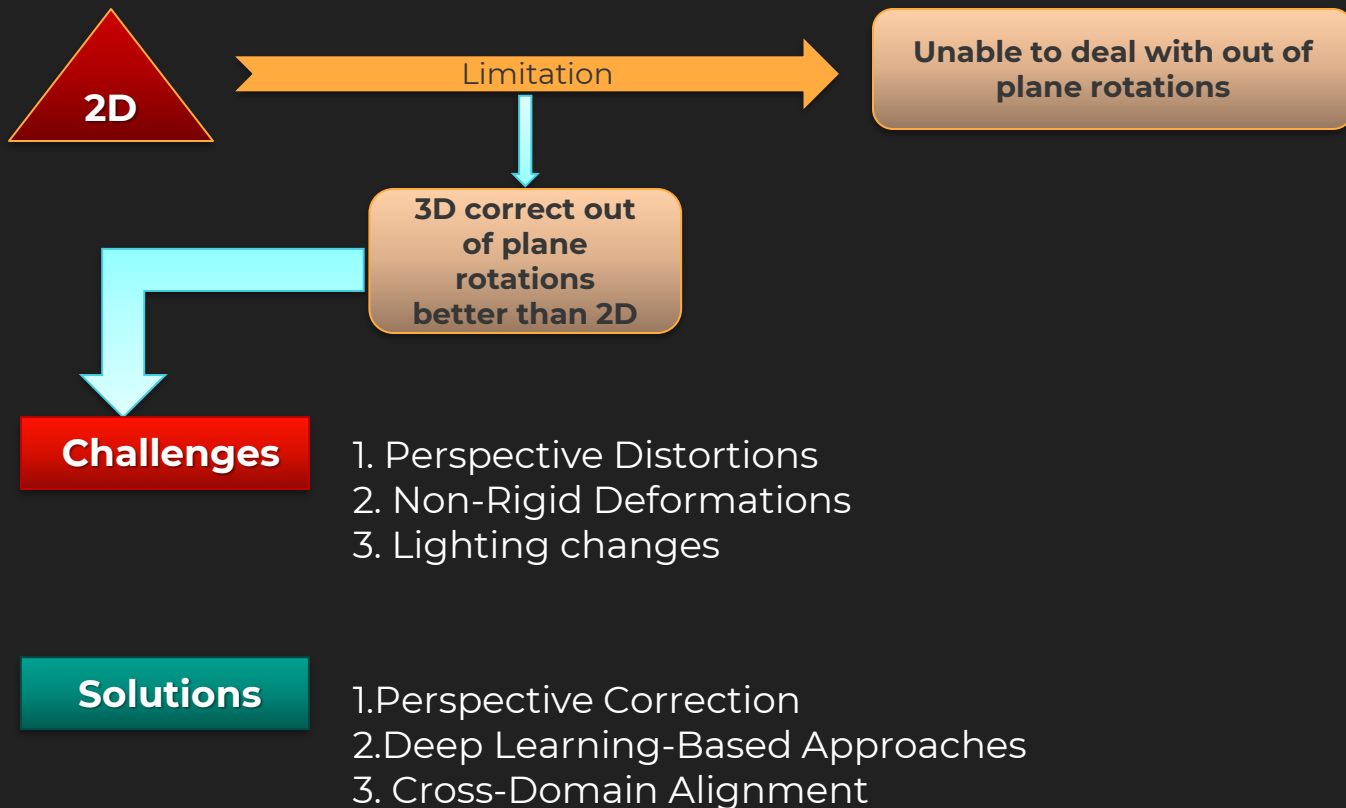
- ✓ This dataset has lower image quality due to motion blur and viewing distance.
- ✓ For training, 50 frame pairs from each video were used to label whether they were the same or not.
- ✓ Reducing errors by over 50% compared to previous methods.
- ✓ After correcting some mislabeled video pairs, the accuracy improved to 92.5%.

Comparison with YTF Dataset



Failures of DeepFace

1. 3D Alignment over 2D Alignment



2. Improving face verification accuracy-

Current approach:

Train on Large Labeled Datasets (e.g., SFC).

Challenge:

Curating these datasets is expensive and time-consuming.

Solution:

Exploring Unsupervised or self-supervised learning techniques could reduce dependence on labeled dataset.

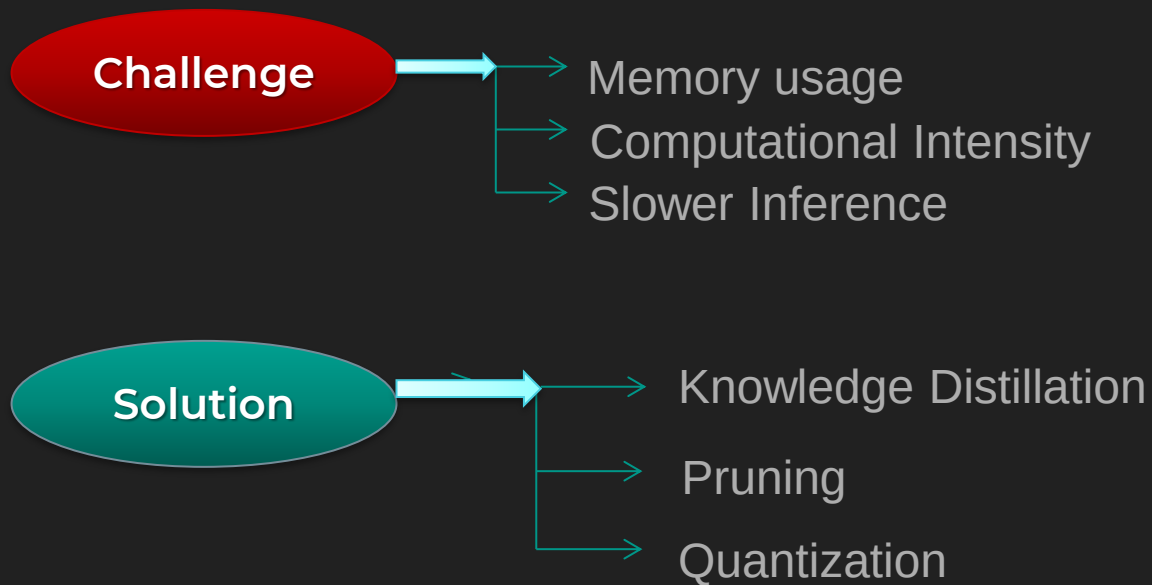
Current V/S Proposed Approach

- Relies on large labeled datasets.
- Requires extensive human labeling.

- Leverages unlabeled data for pre-training.
- Reduces human labeling efforts while maintaining performance

3.DNN's in real time applications

Large DNNs uses significant computational cost and resources both in terms of memory and processing power.



Ctrl C + Ctrl V Crew

Credits:

Rishit
Shubham
Trisha
Shiveish
Chaitanya
Ishwin
Hrishita
Arushi
Vanshika